

Техническая документация программного комплекса «BuilderCV»

Описание: BuilderCV - это интеллектуальная система автоматизированного контроля строительных работ, объединяющая технологии компьютерного зрения (CV), обработки естественного языка (NLP) и потоковой аналитики данных.

1. Модель данных

Основой платформы является реляционная СУБД PostgreSQL. Взаимодействие с базой данных реализовано через ORM SQLAlchemy. Хранение информации организовано в виде связанных сущностей, что обеспечивает консистентность и высокую скорость аналитических выборов.

1.1. Хранение информации о технике и этапах работ

Логика разделена на три ключевые таблицы:

- **Projects (Проекты):** Содержит метаданные строительного объекта (ID, название, дата создания).
- **Stages (Этапы работ):** Хранит детализированную информацию из плана-графика. Включает поля: `start_date` и `end_date` (плановые сроки), `target_volume` (плановый объем) и `current_volume` (фактический выполненный объем). Для связи с модулем видеоаналитики используется поле `nlp_stage_id` — унифицированный тег ID, присваиваемый NLP-моделью.
- **Telemetry (Телеметрия):** Хранит сырые и агрегированные данные с Edge-камер. Поле `detected_objects` (формат JSON) хранит список зафиксированной техники с их `confidence` (уверенностью модели). Поле `calculated_volume` хранит прирост физического объема работ в кубометрах или штуках, а `timestamp` — точную временную метку (ISO 8601).

1.2. Хранение информации об отклонениях и нарушениях

Платформа использует динамическую (on-the-fly) модель расчета отклонений, чтобы избежать рассинхронизации данных:

- **Отставания от графика:** Высчитываются на уровне Frontend-логики. Система сравнивает поле `end_date` (из таблицы Stages) с текущей системной датой. Если срок вышел, а отношение `current_volume / target_volume` меньше 100%, система автоматически генерирует алерт и рассчитывает количество дней просрочки.
- **Недостаток активности (простои):** Анализируется через агрегацию записей в таблице `Telemetry`. Если для активного этапа отсутствует история прироста объема в течение текущего дня, система маркирует этап статусом "Ожидание активности / Простой".

2. Структура кода

Архитектура проекта построена по микросервисному принципу, обеспечивая независимое масштабирование компонентов. Код написан на Python с соблюдением стандартов PEP8.

- **backend/**: Реализует REST API на базе FastAPI. Разделен на логические модули: `models.py` (схемы БД), `schemas.py` (Pydantic-модели валидации), `database.py` (подключение к БД) и `main.py` (эндпоинты контроллеров).
- **frontend/**: UI-интерфейс на базе Streamlit. Включает изолированные компоненты для дашборда (`view_dashboard.py`), рендера диаграммы Ганта (`gantt_component.py`) и генерации видео-отчетов (`media_utils.py`).
- **edge_client/**: Агент компьютерного зрения (`camera_agent.py`), имитирующий работу Edge-устройства на строительной площадке.
- **workers/**: Модули бизнес-логики. Реализуют паттерн "Стратегия" (`strategies.py`), позволяя применять разные математические формулы для различных типов строительных работ.

3. Условия и ограничения

Прототип системы спроектирован с учетом следующих технических допущений:

- **Обучающая выборка и условия освещенности**: Модель YOLO 11s обучена на открытых и синтезированных датасетах строительных площадок. В текущей версии инференс гарантирует высокую точность ($\text{confidence} > 0.5$) при дневном свете и стандартных погодных условиях. Логика обработки видео потока в ночное время или при сильном задымлении/тумане требует дообучения (Fine-tuning) на специфичных данных объекта.
- **Распознаваемые типы техники**: В рамках прототипа словарь классов (Class Mapping) строго ограничен следующей техникой:
 - **Самосвалы** (внутренние классы: `samosval`, `truck`, Самосвал маппятся в единый тег `dump_truck`).
 - **Экскаваторы** (внутренние классы: `excavator`, `ekskavator`, Экскаватор маппятся в единый тег `excavator`).
- **Логика сопоставления (NLP и CV)**:
 - Алгоритм NLP-маппинга (`worker.py`) ищет ключевые корни («котлован», «выемка», «бетон» и так далее) и присваивает этапу соответствующий тег ID.
 - Настроенная математическая модель для данного этапа (EarthworkStrategy) учитывает только детектированные строительные машины, например, 1 распознанный на кадре самосвал = 25 м³ вывезенного грунта. Прочая техника которая не участвует в определении объема работ нужна исключительно для определения тега соответствующего этапа ID

4. Инструкции по сборке, установке и запуску

Для воспроизведения работы прототипа потребуется машина с установленным **Docker**, **Docker Compose** и интерпретатором **Python 3.9+**.

Шаг 1. Развертывание ядра платформы (Backend + Frontend + DB)

1. Склонируйте проект и перейдите в корневую директорию в терминале.
2. Запустите сборку и старт контейнеров следующей командой:

Bash

```
1 | docker-compose up --build -d
```

3. Убедитесь, что контейнеры запущены успешно.

- Веб-интерфейс будет доступен по адресу: `http://localhost:8501`
- API документация (Swagger): `http://localhost:8000/docs`

Шаг 2. Подготовка Edge-клиента (Модуль CV)

1. Откройте новый терминал в корневой папке проекта.
2. Установите зависимости для локального запуска нейросети:

Bash

```
1 | pip install -r requirements.txt
```

3. Убедитесь, что в папке `edge_client` присутствует весовой файл обученной модели (`best.pt`) и директория `demo_frames` с изображениями для симуляции видеопотока.

Шаг 3. Запуск сценария симуляции

1. В браузере (на странице `http://localhost:8501`) откройте боковое меню и загрузите файл тестовый `plan.csv` (находится в корне проекта). Нажмите кнопку «Отправить».
2. Убедитесь, что на Дашборде отобразился статус проекта, диаграмма Ганта и сообщение об ожидании видеопотока.
3. В терминале запустите симулятор агента компьютерного зрения:

Bash

```
1 | python edge_client/camera_agent.py
```

4. Вернитесь в браузер. Как только скрипт начнет отправлять данные распознавания на сервер, интерфейс автоматически обновится: появится раздел активных работ, графики прироста объемов за текущий день и обновится статус на диаграмме.